

STAGE 6: Networking & Local Storage

Flutter & Dart: From Zero to Expert — Stage Textbook 6 of 8

Goal: Connect your app to real APIs and persist data on the device.

Estimated time: 3–4 weeks

Prerequisite: Stage 5 (Navigation & Routing)

6.1 Making HTTP Requests

Add to `pubspec.yaml`: `http: ^1.0.0`

```
import 'package:http/http.dart' as http;
import 'dart:convert';

Future<Map<String, dynamic>> fetchUser(int id) async {
  final response = await http.get(Uri.parse('https://api.example.com/users/$id'));

  if (response.statusCode == 200) {
    return jsonDecode(response.body) as Map<String, dynamic>;
  } else {
    throw Exception('Failed to load user (status ${response.statusCode})');
  }
}
```

Key rules:

- Always check `response.statusCode` before parsing the body.
 - Wrap network calls in `try/catch` at the call site to handle timeouts and connectivity errors gracefully.
-

6.2 JSON Serialization with Models

```
class User {
  final int id;
  final String name;
  final String email;

  User({required this.id, required this.name, required this.email});

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json['id'] as int,
      name: json['name'] as String,
      email: json['email'] as String,
    );
  }

  Map<String, dynamic> toJson() => {'id': id, 'name': name, 'email': email};
}

Future<User> fetchUser(int id) async {
  final response = await http.get(Uri.parse('https://api.example.com/users/$id'));
  return User.fromJson(jsonDecode(response.body));
}
```

Why it matters: Converting raw JSON into typed Dart classes (`fromJson` / `toJson`) catches mistakes at compile time and makes the rest of your app work with safe, predictable objects instead of loose `Map<String,`

`dynamic>` values.

6.3 Showing Loading, Error, and Success States

```
class UserScreen extends StatefulWidget {
  const UserScreen({super.key});
  @override
  State<UserScreen> createState() => _UserScreenState();
}

class _UserScreenState extends State<UserScreen> {
  late Future<User> _futureUser;

  @override
  void initState() {
    super.initState();
    _futureUser = fetchUser(1);
  }

  @override
  Widget build(BuildContext context) {
    return FutureBuilder<User>(
      future: _futureUser,
      builder: (context, snapshot) {
        if (snapshot.connectionState == ConnectionState.waiting) {
          return const Center(child: CircularProgressIndicator());
        } else if (snapshot.hasError) {
          return Center(child: Text('Error: ${snapshot.error}'));
        } else {
          final user = snapshot.data!;
          return Center(child: Text(user.name));
        }
      },
    );
  }
}
```

Key rule: `FutureBuilder` is the standard way to drive UI directly from an async call — it has three states you must always handle: waiting, error, and data.

6.4 Simple Local Storage: SharedPreferences

Add to `pubspec.yaml`: `shared_preferences: ^2.0.0`

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> setUsername(String name) async {
  final prefs = await SharedPreferences.getInstance();
  await prefs.setString('username', name);
}

Future<String?> loadUsername() async {
  final prefs = await SharedPreferences.getInstance();
  return prefs.getString('username');
}
```

Use case: small key-value data — settings, flags like "has seen onboarding", auth tokens (encrypted versions for sensitive data), and simple preferences.

6.5 Structured Local Storage: sqflite and Hive

sqflite (SQL-based, good when you need queries and relationships):

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

Future<Database> openAppDb() async {
  final path = join(await getDatabasesPath(), 'app.db');
  return openDatabase(
    path,
    version: 1,
    onCreate: (db, version) => db.execute(
      'CREATE TABLE notes(id INTEGER PRIMARY KEY, text TEXT)',
    ),
  );
}

Future<void> insertNote(Database db, String text) async {
  await db.insert('notes', {'text': text});
}
```

Hive (NoSQL, key-value, faster for simple object storage):

```
import 'package:hive_flutter/hive_flutter.dart';

await Hive.initFlutter();
var box = await Hive.openBox('notesBox');
box.put('note1', 'Buy groceries');
print(box.get('note1'));
```

Choosing between them: use `SharedPreferences` for small flags/settings, `Hive` for fast key-value object storage without SQL, and `sqflite` when you genuinely need relational queries (joins, filters across tables).

6.6 Hands-On Project: Weather Lookup App with Caching

Build a Flutter app that:

1. Calls a public weather API (e.g., Open-Meteo, which needs no API key) using `http`, and parses the JSON into a `Weather` model with `fromJson`.
2. Displays the result through a `FutureBuilder` with proper loading and error states.
3. Caches the last successful result using `SharedPreferences` (store the JSON string), and shows that cached result immediately on app start while the live request is still in flight.
4. Has a "Clear Cache" button that removes the saved data.

This project combines HTTP requests, JSON models, `FutureBuilder` state handling, and local persistence — all of Stage 6.

6.7 Stage 6 Test

Question 1: Why should you always check `response.statusCode` before parsing an HTTP response body?

Question 2: What is the purpose of a `fromJson` factory constructor?

Question 3: What are the three states a `FutureBuilder` must handle, and what triggers each one?

Question 4: When would you choose `Hive` over `SharedPreferences`?

Question 5: Name one situation where `sqflite` (a relational database) is clearly the better choice over

`Hive` or `SharedPreferences` .

Passing score: Get all 5 correct before moving to Stage 7.